



UNIVERSITÉ

Projet de bioinformatique

Déployer et comprendre AlphaFold2

Auteur :
Guillaume ROSIN

Enseignant :
Prénom NOM

Résumé

AlphaFold2 prédit la structure tridimensionnelle d'une protéine à partir de sa seule séquence, en exploitant les signaux de coévolution contenus dans un alignement multiple de séquences homologues. Ce projet met en place un workflow reproductible *séquence* → *AlphaFold2* → *structure* → *analyse*. Après comparaison de plusieurs solutions, nous avons retenu ColabFold 1.5.5 exécuté localement dans Docker : il utilise les mêmes poids qu'AlphaFold2, mais remplace les 2,6 To de bases de données par un serveur MMseqs2, et fonctionne sans GPU. Le workflow a été testé sur la calmoduline humaine (UniProt P0DP23, 149 résidus), prédite en 1 h 57 sur CPU. Le pLDDT (moyenne 82,8) montre deux lobes bien prédits reliés par une région centrale moins fiable. La PAE révèle que, si chaque lobe est rigide (environ 4–5 Å), leur orientation relative reste incertaine (environ 20 Å). Un `README.md` et deux scripts permettent de refaire une prédiction et son analyse en deux commandes.

24 septembre 2026

1 AlphaFold2 : principe général

Le problème. La séquence d'acides aminés d'une protéine détermine sa structure tridimensionnelle [2], mais le nombre de conformations possibles est tel qu'une exploration exhaustive est impossible. On appelle cela le « problème du repliement » [5]. Plutôt que de simuler le repliement physique, AlphaFold2 [7] *apprend*, à partir des structures connues de la PDB, la relation entre séquence et structure.

Entrée et sortie. L'utilisateur fournit uniquement la **séquence** de la protéine (format FASTA). AlphaFold2 renvoie les **coordonnées 3D** de tous les atomes lourds (fichier PDB ou mmCIF), accompagnées de **mesures de confiance**.

Le MSA et la coévolution. La première étape consiste à rechercher, dans de grandes bases de séquences, des protéines *homologues* (issues d'un ancêtre commun) et à les aligner : c'est l'**alignement multiple de séquences** (MSA). Chaque ligne est une séquence homologue et chaque colonne une position équivalente de la protéine. Le MSA apporte deux informations structurales :

- la **conservation** : une position qui ne varie jamais est probablement essentielle (cœur hydrophobe, site actif) ;
- la **coévolution** : si deux résidus sont en contact, une mutation de l'un est souvent compensée par une mutation de l'autre, sinon la protéine perd sa structure. Deux colonnes qui varient *ensemble* indiquent donc deux résidus probablement proches dans l'espace [5].

Plus le MSA est profond et diversifié, plus ces signaux sont fiables. À l'inverse, une protéine « orpheline », sans homologues, est en général mal prédite.

Les grandes étapes. La figure 1 résume le processus :

1. **Recherche d'homologues** dans les bases de séquences (MSA) et, en option, de structures connues similaires (*gabarits*) ;
2. **Evoformer** : un réseau de neurones fait circuler l'information entre le MSA et une « représentation de paires », qui décrit pour chaque couple de résidus (i, j) leur relation spatiale probable ;
3. **Module de structure** : il transforme la représentation de paires en coordonnées 3D (squelette, puis chaînes latérales) ;
4. **Recyclage** : la structure obtenue est réinjectée dans le réseau (3 fois par défaut) pour l'affiner.

Pourquoi des mesures de confiance ? Une prédiction n'est pas une mesure expérimentale : sa qualité varie d'une protéine à l'autre, et même d'une région à l'autre. AlphaFold2 a été entraîné à *estimer sa propre erreur*. Il produit un score local par résidu, le **pLDDT**, et une erreur relative entre paires de résidus, la **PAE** (section 5). Sans ces scores, l'utilisateur ne pourrait pas distinguer une région fiable d'une région inventée.

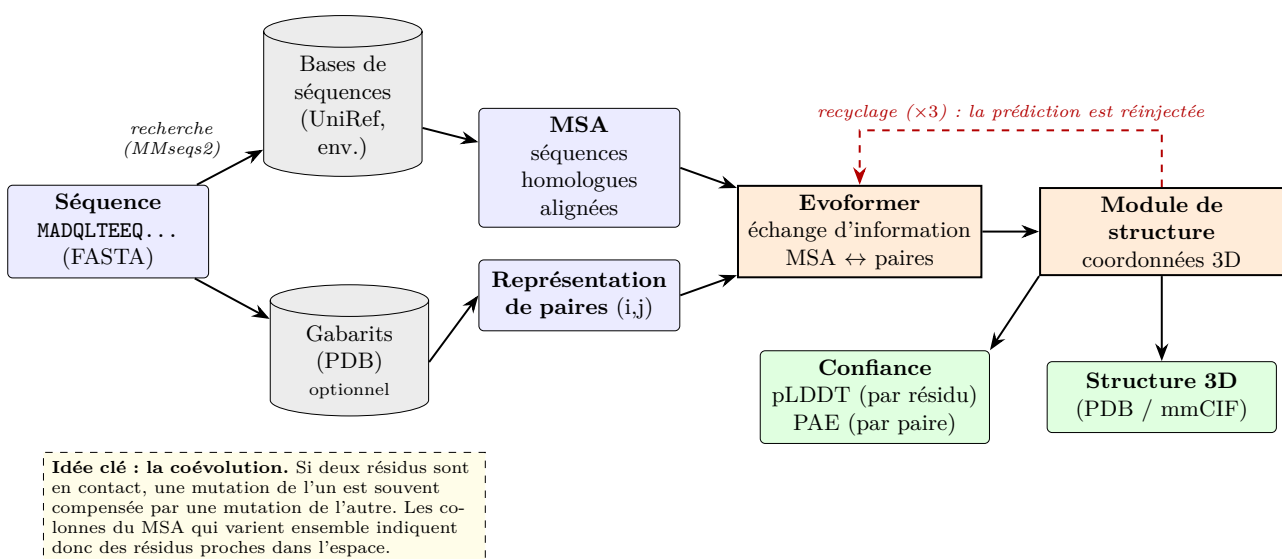


FIGURE 1 – Schéma du fonctionnement général d'AlphaFold2 (réalisation personnelle, d'après [7, 5]).

2 Choix technique

Le tableau 1 compare les solutions sérieusement envisagées. La machine disponible est un PC Linux (Debian 12, 12 cœurs, 14 Go de RAM, **sans GPU NVIDIA**) sur lequel Docker est installé.

	Installation officielle AlphaFold2 [6]	ColabFold sur Google Colab [10]	ColabFold local (Docker) [9]	AlphaFold DB / AlphaFold Server [14, 1]
À installer	Docker, pilotes NVIDIA, scripts DeepMind	Rien (navigateur, compte Google)	Docker + image ColabFold (~10 Go) + poids (~5 Go)	Rien (site web)
Calcul	Local	Serveurs Google (GPU gratuit)	Local (CPU ou GPU)	Serveurs EBI / Google
Ressources	GPU recommandé, $\geq$ 64 Go RAM	Aucune localement	CPU suffisant pour < 300 résidus ; GPU optionnel	Aucune
Bases de données	~2,6 To à télécharger (UniRef, BFD, MGnify, PDB...)	Aucune : MSA via le serveur MMseqs2 public	Aucune : MSA via le serveur MMseqs2 public	Aucune
Avantages	Pipeline de référence, hors ligne	Très simple, rapide (GPU)	Reproductible (version figée), scriptable, pas de limite de session	Instantané
Inconvénients	Irréaliste sur notre machine (disque, GPU)	Session limitée, GPU non garanti, dépend de Google	Lent sur CPU ; Internet requis pour le MSA	DB : protéines déjà prédites seulement ; Server = AlphaFold3, pas AF2

TABLE 1 – Comparaison des solutions envisagées pour exécuter AlphaFold2.

Solution retenue : ColabFold 1.5.5 exécuté localement dans Docker , avec Google Colab documenté comme alternative dans le `README.md`. ColabFold [10] utilise *exactement les mêmes réseaux et les mêmes poids* qu’AlphaFold2. Il remplace seulement la construction du MSA (HHblits/JackHMMER sur 2,6 To de bases) par une recherche MMseqs2 [11] sur un serveur public, qui prend quelques secondes au lieu de plusieurs heures. Ce choix supprime les deux obstacles de l’installation officielle, le disque et le GPU, tout en gardant un calcul local, reproductible et automatisable. Docker fige toutes les dépendances (Python, JAX, TensorFlow) : un étudiant de l’année prochaine obtiendra le même environnement avec une seule commande. Google Colab reste la voie la plus simple pour ceux qui n’ont ni Linux ni Docker, mais nous ne l’avons pas retenu comme solution principale : les sessions sont limitées dans le temps, l’accès au GPU n’est pas garanti et le notebook évolue sans versionnage explicite.

3 Mise en œuvre

Environnement. Debian GNU/Linux 12 (noyau 6.12), processeur 12 cœurs, 14 Go de RAM, pas de GPU ; Docker 29.8.1 ; Python 3.11.2 pour l’analyse (numpy 2.4, matplotlib 3.11, dans un environnement virtuel). Aucun compte n’est nécessaire ; seule une connexion Internet est requise, pour le MSA et le téléchargement initial.

Étapes. Les commandes exactes figurent dans le `README.md` ; en résumé :

1. `docker pull ghcr.io/sokrypton/colabfold:1.5.5-cuda12.2.2`, pour une version figée de ColabFold ;
2. `python -m colabfold.download` dans le conteneur : téléchargement des poids AlphaFold2 (5,3 Go) dans un dossier `cache/` monté en volume, pour ne les télécharger qu’une fois ;
3. `colabfold_batch -num-models 5 séquence.fasta results/` : MSA via MMseqs2, puis prédiction par les 5 modèles AlphaFold2 (`alphafold2_ptm`) ;
4. `scripts/analyse.py` : extraction du pLDDT et de la PAE, et production des figures.

Les étapes 2 et 3 sont regroupées dans le script `scripts/predict.sh`, qui détecte automatiquement la présence d’un GPU.

Difficultés rencontrées et solutions.

- **Pas de GPU** : l'image est compilée pour CUDA, mais JAX bascule automatiquement sur le CPU (message `no GPU detected, will be using CPU`). Le calcul fonctionne, mais il est lent : 1 h 57 au total pour 149 résidus et 5 modèles, soit environ 23 min par modèle (voir section 4).
- **Fichiers appartenant à root** : par défaut, le conteneur écrit ses résultats en tant que root. On lance donc Docker avec `-user $(id -u):$(id -g) -e HOME=/tmp`.
- **Noms de fichiers très longs** : ColabFold nomme les sorties d'après l'en-tête FASTA complet (une centaine de caractères, par exemple `sp_P0DP23_CALM1_...`). Il faut raccourcir l'en-tête (`>P0DP23`) avant de lancer le calcul.
- **Mémoire** : TensorFlow affiche des avertissements `Allocation exceeds 10% of free system memory`. Ils sont sans conséquence ici : la consommation plafonne à environ 4 Go.
- **Modèle de rapport** : `inputenc[utf8x]` et un `\lstset` faisant référence à des macros non définies empêchaient la compilation ; nous les avons remplacés par `utf8`.

4 Test sur une protéine réelle

Protéine choisie. Nous avons choisi la **calmoduline-1 humaine** (*Homo sapiens*, gène *CALM1*), identifiant UniProtKB **P0DP23** [13, 12], de **149 résidus**. Voici la séquence utilisée, telle que fournie par UniProt (fichier `data/P0DP23.fasta`) :

```
MADQLTEEQIAEFKEAFSLFDKDGDTITTKELGTVMRSLGQNPTEAELQDMINEVDADG
NGTIDFPEFLTMMARKMKDTSDEEIREAFRVFDKDGNGYISAAELRHVMTNLGEKLTDE
EVDEMIREADIDGGQVNYEEFVQMMTAK
```

Ce choix a trois raisons. D'abord, la taille est compatible avec un calcul sur CPU. Ensuite, la protéine est très étudiée et sa structure expérimentale est connue (par exemple PDB 1CLL [4]). Enfin, et surtout, sa structure en « haltère » se prête bien à l'analyse des scores de confiance : deux lobes globulaires (chacun formé de deux motifs EF-hand fixant le Ca^{2+}) sont reliés par une région centrale connue pour être flexible en solution [3]. On s'attend donc à ce que le pLDDT et la PAE ne racontent pas la même chose.

Exécution. La commande `colabfold_batch -num-models 5` a été lancée avec les paramètres par défaut : MSA `mmseqs2_uniref_env`, pas de gabarits, pas de relaxation Amber, 3 recyclages. Le MSA, construit en environ 15 s, contient **21 884 séquences**. Chacun des 5 modèles a demandé environ 23 min sur CPU, soit **1 h 57 au total**. Le tableau 2 présente les fichiers importants produits dans `results/P0DP23/`.

Fichier	Contenu
<code>*_unrelaxed_rank_001_*.pdb</code>	Structure prédite (format PDB) du meilleur modèle ; le pLDDT est stocké dans la colonne <i>B-factor</i>
<code>*_scores_rank_001_*.json</code>	Clés <code>plddt</code> (149 valeurs), <code>pae</code> (matrice 149×149 en Å), <code>ptm</code>
<code>*_rank_002 ...005_*</code>	Les 4 autres modèles, classés par pLDDT moyen décroissant
<code>*.a3m, *_coverage.png</code>	MSA utilisé et profondeur de l'alignement par position
<code>log.txt, config.json</code>	Journal et paramètres exacts, pour la traçabilité

TABLE 2 – Principaux fichiers produits par ColabFold.

Structure obtenue. Le meilleur modèle (modèle 4) a un pLDDT moyen de 82,8 et un pTM de 0,487. Les 5 modèles sont très proches : pLDDT moyens entre 78,6 et 82,8, pTM entre 0,458 et 0,487. La structure (figure 2) présente bien deux lobes compacts d'hélices α , dont les centres sont distants de 32,6 Å, reliés par une région centrale étirée.

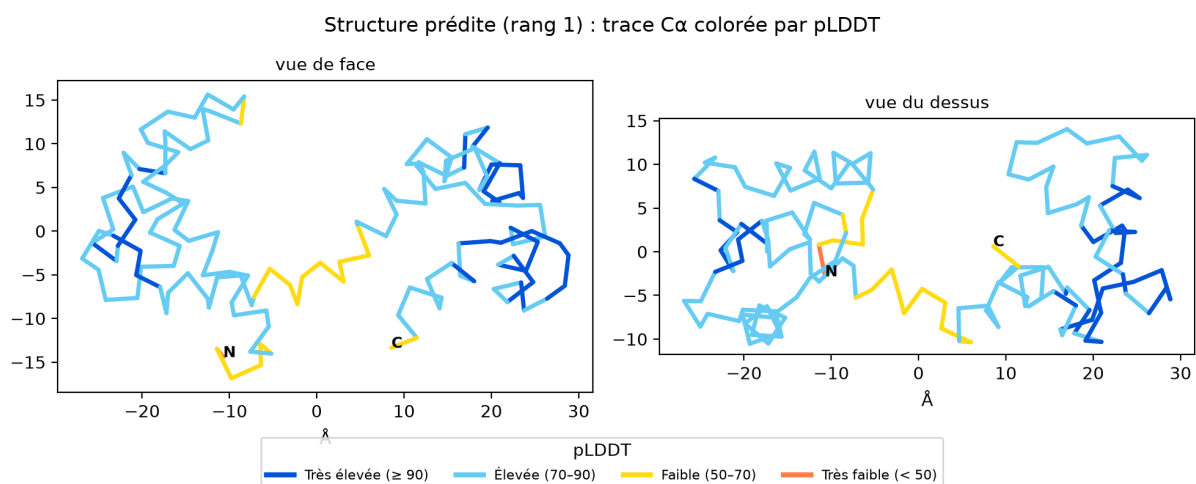


FIGURE 2 – Trace C α du meilleur modèle, colorée par pLDDT (produite par `scripts/analyse.py`). Elle peut aussi être visualisée en 3D en déposant le fichier PDB sur <https://molstar.org/viewer/>.

5 Analyse des scores de confiance

Les deux scores se trouvent dans le fichier `*_scores_rank_001_*.json`. Le script `scripts/analyse.py` le lit avec le module `json` de Python et produit les figures ainsi qu'un résumé chiffré (`figures/PODP23/resume.txt`). Il vérifie aussi que le pLDDT lu dans le JSON est identique à celui stocké dans la colonne B-factor du PDB (écart maximal : 0,00).

5.1 pLDDT : confiance locale

Définition. Le pLDDT (*predicted Local Distance Difference Test*) est une note de 0 à 100 attribuée à *chaque résidu*. Il estime le score lDDT-C α [8] que la prédiction obtiendrait si on la comparait à la vraie structure. Autrement dit, il indique si les distances entre ce résidu et ses voisins proches sont correctes. Ce score est *local* et ne nécessite aucune superposition [7, 5].

- > 90 : très fiable, squelette et chaînes latérales compris ;
- **70–90** : squelette fiable, chaînes latérales parfois mal placées ;
- **50–70** : faible confiance ;
- < 50 : à ne pas interpréter. C'est souvent une région intrinsèquement désordonnée, mais cela peut aussi signifier que l'information manque, par exemple un linker entre deux domaines.

Le pLDDT sert donc à décider quelles parties de la structure on peut exploiter. En revanche, un pLDDT élevé partout *ne garantit pas* que les domaines sont correctement placés les uns par rapport aux autres [5].

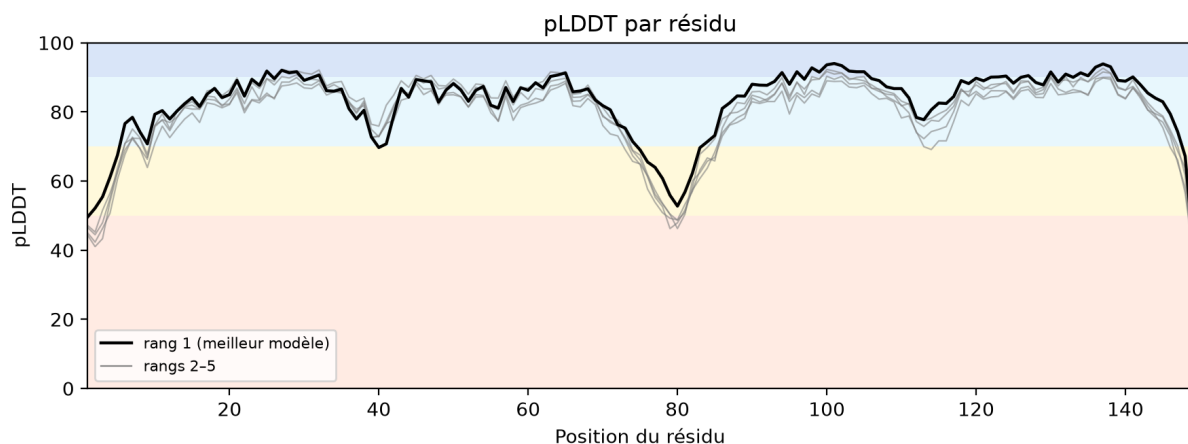


FIGURE 3 – pLDDT par résidu du meilleur modèle (noir) et des 4 autres (gris). Fonds colorés : classes de confiance.

Résultats. La figure 3 montre que 88,6 % des résidus ont un pLDDT supérieur à 70 : 21,5 % au-dessus de 90 et 67,1 % entre 70 et 90. Les deux lobes (environ 6–72 et 86–145) sont prédits avec confiance, avec des maxima au-dessus de 90 dans les hélices. Trois zones descendent sous 70 :

- **la région centrale 75–83** (minimum de 53 au résidu 80) : c’est le linker entre les deux lobes, dont la flexibilité en solution est connue [3] ;
- **les extrémités 1–5 et 148–149** (minimum de 43 au résidu C-terminal) : les extrémités libres d’une chaîne sont typiquement peu contraintes ;
- un léger creux vers le résidu 40, et un autre autour de 112–113 (environ 78), à la jonction entre les deux motifs EF-hand de chaque lobe.

Les 5 modèles donnent le même profil (courbes grises), ce qui montre que ces variations ne sont pas dues au hasard d’un seul modèle.

5.2 PAE : confiance dans les positions relatives

Définition. La PAE (*Predicted Aligned Error*) est une *matrice* $N \times N$. La valeur (i,j) est l’erreur attendue, en ångströms, sur la position du résidu i si l’on superpose la prédiction et la vraie structure sur le résidu j [5]. Les deux axes représentent donc les résidus de la protéine : en ordonnée, le résidu dont on évalue la position ; en abscisse, le résidu sur lequel on aligne. La diagonale, qui correspond à l’alignement d’un résidu sur lui-même, vaut toujours presque zéro et ne porte pas d’information. Une valeur faible (vert foncé) signifie que la position relative des deux résidus est fiable ; une valeur élevée (vert clair) signifie qu’elle ne l’est pas.

Différence avec le pLDDT. Le pLDDT répond à la question « ce résidu est-il bien placé par rapport à ses voisins ? » ; la PAE répond à la question « la position de ce résidu *par rapport à ce résidu éloigné* est-elle fiable ? ». La PAE est donc indispensable pour juger l’**arrangement des domaines**.

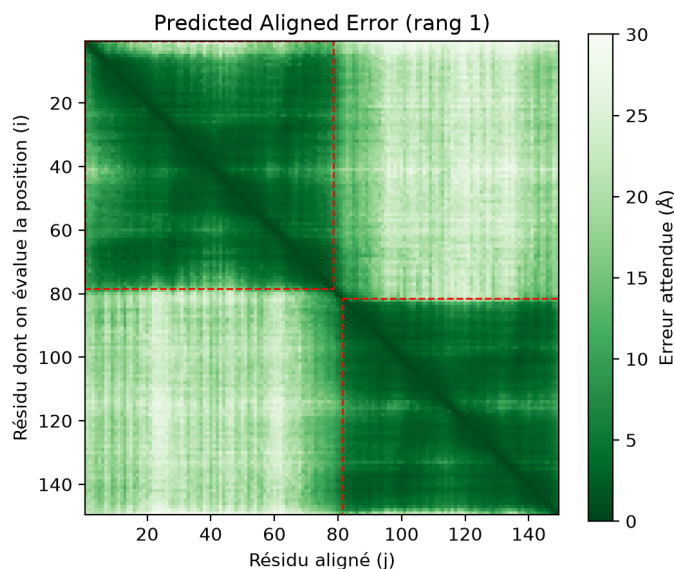


FIGURE 4 – Matrice PAE du meilleur modèle. Cadres rouges : lobe N (1–78) et lobe C (82–149).

Résultats. La matrice (figure 4) montre **deux blocs vert foncé** le long de la diagonale, qui correspondent aux deux lobes. La PAE moyenne vaut 5,1 Å à l’intérieur du lobe N et 3,8 Å à l’intérieur du lobe C : chaque lobe est une unité rigide bien prédite. En revanche, les **blocs hors diagonale** sont clairs, avec une PAE moyenne de 20,6 et 19,7 Å entre les deux lobes. AlphaFold2 est donc sûr de la forme de chaque lobe, mais *pas* de leur orientation l’une par rapport à l’autre.

C’est l’exemple typique où le pLDDT seul induirait en erreur. Hormis le linker, presque tous les résidus ont un pLDDT supérieur à 70, et pourtant la position relative des lobes dans le fichier PDB n’est qu’une conformation parmi d’autres possibles. Ce résultat est cohérent avec le pTM modeste (0,49) et avec la flexibilité connue de la calmoduline, dont la région centrale se replie pour envelopper ses peptides cibles. Enfin, AlphaFold2 ignore les ions : la structure est prédite sans les quatre Ca^{2+} qui, dans la cellule, modifient la conformation des lobes.

6 Retour d'expérience

Ce qui a fonctionné. L'image Docker de ColabFold a fonctionné du premier coup sur une machine sans GPU. Le serveur MMseqs2 a construit le MSA en quelques secondes. Le point essentiel est que toute la chaîne, de la séquence à la structure puis aux figures, tient en deux commandes (`predict.sh` puis `analyse.py`), sans installation de bases de données.

Ce qui a posé problème. Le temps de calcul sur CPU (1 h 57 pour 149 résidus) est le principal inconvénient. Il devient prohibitif au-delà de quelques centaines de résidus, car le coût croît à peu près comme le carré de la longueur. Viennent ensuite des détails pratiques (droits des fichiers, noms trop longs) qui ne sont documentés nulle part de façon centralisée.

Recommandations.

- Sans GPU et pour une protéine de plus de 300 résidus, utiliser le notebook ColabFold sur Google Colab, qui produit exactement les mêmes fichiers ; le script d'analyse s'applique tel quel.
- Pour un premier essai, réduire le calcul avec `-num-models 1`.
- Toujours lire le pLDDT *et* la PAE avant d'interpréter la structure, et ne jamais conclure sur l'orientation relative de deux domaines sans consulter la PAE.
- Le serveur MMseqs2 est une ressource partagée : ne pas soumettre des centaines de séquences d'un coup.

Références

- [1] Josh Abramson, Jonas Adler, Jack Dunger, et al. Accurate structure prediction of biomolecular interactions with AlphaFold 3. *Nature*, 630 :493–500, 2024.
- [2] Christian B. Anfinsen. Principles that govern the folding of protein chains. *Science*, 181 :223–230, 1973.
- [3] Gaetano Barbato, Mitsuhiro Ikura, Lewis E. Kay, Richard W. Pastor, and Ad Bax. Backbone dynamics of calmodulin studied by ^{15}N relaxation using inverse detected two-dimensional NMR spectroscopy : the central helix is flexible. *Biochemistry*, 31(23) :5269–5278, 1992.
- [4] Rajagopal Chattopadhyaya, William E. Meador, Anthony R. Means, and Florante A. Quiocho. Calmodulin structure refined at 1.7 Å resolution. *Journal of Molecular Biology*, 228 :1177–1192, 1992.
- [5] EMBL-EBI Training. AlphaFold : A practical guide. <https://www.ebi.ac.uk/training/online/courses/alphafold/>. Pages « What is the protein folding problem? », « A high-level overview », « pLDDT : Understanding local confidence », « PAE : A measure of global confidence », « Strengths and limitations of AlphaFold 2 ». Consulté le 24/09/2026.
- [6] Google DeepMind. AlphaFold – dépôt officiel de google deepmind. <https://github.com/google-deepmind/alphafold>. Section « Genetic databases » : besoins en disque et en GPU de l’installation complète.
- [7] John Jumper, Richard Evans, Alexander Pritzel, et al. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596 :583–589, 2021. <https://doi.org/10.1038/s41586-021-03819-2>.
- [8] Valerio Mariani, Marco Biasini, Alessandro Barbato, and Torsten Schwede. IDDT : a local superposition-free score for comparing protein structures and models using distance difference tests. *Bioinformatics*, 29(21) :2722–2728, 2013.
- [9] Milot Mirdita, Sergey Ovchinnikov, and Martin Steinegger. ColabFold – dépôt officiel (v1.5.5). <https://github.com/sokrypton/ColabFold>. Image Docker ghcr.io/sokrypton/colabfold:1.5.5-cuda12.2.2; notebook <https://colab.research.google.com/github/sokrypton/ColabFold/blob/main/AlphaFold2.ipynb>.
- [10] Milot Mirdita, Konstantin Schütze, Yoshitaka Moriwaki, Lim Heo, Sergey Ovchinnikov, and Martin Steinegger. ColabFold : making protein folding accessible to all. *Nature Methods*, 19 :679–682, 2022. <https://doi.org/10.1038/s41592-022-01488-1>.
- [11] Martin Steinegger and Johannes Söding. MMseqs2 enables sensitive protein sequence searching for the analysis of massive data sets. *Nature Biotechnology*, 35 :1026–1028, 2017.
- [12] The UniProt Consortium. UniProt : the universal protein knowledgebase in 2023. *Nucleic Acids Research*, 51(D1) :D523–D531, 2023.
- [13] UniProt. UniProtKB P0DP23 (CALM1_HUMAN) – calmodulin-1. <https://www.uniprot.org/uniprotkb/P0DP23>. Séquence FASTA : <https://rest.uniprot.org/uniprotkb/P0DP23.fasta>.
- [14] Mihaly Varadi, Stephen Anyango, Mandar Deshpande, et al. AlphaFold protein structure database : massively expanding the structural coverage of protein-sequence space with high-accuracy models. *Nucleic Acids Research*, 50(D1) :D439–D444, 2022.